PHYS 2550 - Homework 2

All code and outputs: answers.ipynb on GitHub.

Part 1 - Theory

T1. Counting parameters

I counted biases using $(n_{\text{in}} + 1)n_{\text{out}}$ for dense layers and $(k_h k_w C_{\text{in}} + 1)C_{\text{out}}$ for convolutions. Dense layers are numbered within each model.

Layer	CNN	FCNN-matched	FCNN-large
Conv 1	448	–	–
Conv 2	4,640	–	–
Conv 3	18,496	–	–
Dense 1	147,520	165,912	1,769,728
Dense 2	650	1,600	32,896
Dense 3	–	650	1,290
Total	**171,754**	**168,162**	**1,803,914**

ReLU, pooling, flattening, and dropout add no parameters. The CNN's first dense layer holds $147520/171754 \approx 85.9\%$ of the total. FCNN-matched has $6977h + 714$ parameters: 24 nodes fit the budget, but 25 give 175,139. The CNN saves parameters by reusing filters.

T2(a). Convolution by hand

Each output is five times the center minus its four neighbors. For example, the top-left value is $5(3) - 2 - 0 - 1 - 1 = 11$. The output is 3×3:

$$I * K = \begin{pmatrix} 11 & -4 & 6 \\ -4 & 15 & -4 \\ -5 & 2 & 9 \end{pmatrix}.$$

T2(b). Output sizes

For input size n, kernel k, padding p, and stride s,

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

Padding 1 and stride 1 give 5×5. Stride 2 without padding gives 2×2.

T2(c). Pooling

Using 2×2 windows with stride 1:

$$P_{\max} = \begin{pmatrix} 15 & 15 \\ 15 & 15 \end{pmatrix}, \qquad P_{\text{avg}} = \begin{pmatrix} 4.5 & 3.25 \\ 2 & 5.5 \end{pmatrix}.$$

Moving the bright pixel moves the response. Max pooling can stay unchanged if the peak stays in its window; average pooling uses all four values. Neither is always unchanged by a shift.

T2(d). Background and noise

The weights sum to 1, so a constant background stays unchanged. For independent noise,

$$\mathrm{Var}(I * K) = \sigma^2 \sum_{ij} K_{ij}^2 = 29\sigma^2.$$

The noise standard deviation grows by $\sqrt{29} \approx 5.39$. Sharpening can therefore make noise look like features in faint galaxies.

T2(e). Cross-correlation

This kernel is unchanged by a 180° rotation. A learned kernel can just learn the flipped weights, so either convention can represent the same filters.

T3(a). Shapes through the CNN

Shapes are height $\times$ width $\times$ channels. Layers grouped together have the same output shape.

Layer	Output shape
Input	$48 \times 48 \times 3$
Conv 1, ReLU	$48 \times 48 \times 16$
Pool 1	$24 \times 24 \times 16$
Conv 2, ReLU	$24 \times 24 \times 32$
Pool 2	$12 \times 12 \times 32$
Conv 3, ReLU	$12 \times 12 \times 64$
Pool 3	$6 \times 6 \times 64$
Flatten	2304
Dense 64, ReLU, dropout	64
Dense 10	10 logits

T3(b). Receptive field

Starting at $(r, j) = (1, 1)$, the receptive-field width and spacing update as $r' = r + (k-1)j$ and $j' = sj$.

Layer	Conv 1	Pool 1	Conv 2	Pool 2	Conv 3	Pool 3
r	3	4	8	10	**18**	22
j	1	2	2	4	4	8

The third convolution sees 18×18 pixels, less than a 20–40-pixel galaxy. The last pool sees 22 pixels, enough for a small galaxy in the right position. The dense layer combines information across the whole image.

T3(c). Strided first convolution

With stride 2 and no padding, $\lfloor (48-3)/2 \rfloor + 1 = 23$, giving $23 \times 23 \times 16$.

T4(a). Equivariance and invariance

Equivariance means the output shifts with the input; invariance means it stays unchanged. With $(T_\delta x)(u) = x(u - \delta)$, these are $f(T_\delta x) = T_\delta f(x)$ and $f(T_\delta x) = f(x)$. Ignoring boundaries, a stride-1 convolution gives

$$f(T_\delta x)(u) = b + \sum_v K(v) x(u + v - \delta) = f(x)(u - \delta) = T_\delta f(x)(u).$$

T4(b). Approximate invariance

Pooling can keep a peak when it moves within a window. Crossing windows, padding at the edges, and the dense layer's different weights at different positions stop this from being exact invariance.

T4(c). Shift prediction

An FCNN has separate weights at each position, so neither property is built in. I expect shifts to reduce accuracy for all models, but less for the CNN. Large shifts also cut off light.

T5. Scrambled pixels

If the input becomes xP, the FCNN can use first-layer weights P^TW, since

$$(xP)(P^TW) = xW.$$

I expect similar FCNN accuracy after retraining. The CNN assumes nearby pixels form useful patterns; scrambling breaks these, so I expect it to do worse. The same shuffle must be used for every image and all channels.

T6(a). Forward-pass MACs

I count one multiply–accumulate as one MAC, leaving out biases, activations, and pooling comparisons.

CNN layer	MACs per image
Conv 1: $48^2 \cdot 16 \cdot 27$	995,328
Conv 2: $24^2 \cdot 32 \cdot 144$	2,654,208
Conv 3: $12^2 \cdot 64 \cdot 288$	2,654,208
Dense 64: $2304 \cdot 64$	147,456
Dense 10: $64 \cdot 10$	640
Total	**6,451,840**

FCNN-matched uses $6912 \cdot 24 + 24 \cdot 64 + 64 \cdot 10 = 168,064$ MACs. FCNN-large uses $6912 \cdot 256 + 256 \cdot 128 + 128 \cdot 10 = 1,803,520$.

T6(b). Cost per step

The CNN uses $6451840/168064 \approx 38.4$ times as many forward MACs. Backpropagation adds similar-order work. Shared weights still get applied at many positions, so fewer parameters don't mean less work. The time ratio depends on hardware.

T6(c). Training speed

A CNN might need fewer epochs to reach the same accuracy, but each epoch costs more. I would compare seconds per epoch, epochs and total time to a common validation target, and which runs fail to reach it.

T7(a). Average pooling

For four independent pixels, $A = (X_1 + X_2 + X_3 + X_4)/4$:

$$\mathbb{E}[A] = \mu, \qquad \mathrm{Var}(A) = \frac{4\sigma^2}{16} = \frac{\sigma^2}{4}, \qquad \sigma_A = \frac{\sigma}{2}.$$

T7(b). Max pooling

It picks upward fluctuations, so it is biased high. For Gaussian noise, $\mathbb{E}[\max_i X_i] \approx \mu + 1.03\sigma$.

T7(c). Choice of pooling

I would try average pooling for faint galaxies: it reduces independent noise while keeping their spread-out light. Max pooling can pick noise spikes. For a dark object on a bright background, I would use min pooling.

T8(a). Training curves

FCNN-large starts overfitting around **epoch 8**, which early stopping would keep. Patience 5 would stop training about five epochs later. The CNN shows no clear overfitting or underfitting; a plateau alone doesn't show underfitting.

T8(b). Reducing the gap

I would try weight decay to discourage large weights, plus rotations and flips so the model sees more valid orientations. I would tune these using validation performance.

T8(c). Smaller CNN gap

Shared local filters give the CNN less freedom to memorize individual pixels, and pooling helps with shifts. It also has fewer parameters than FCNN-large, so FCNN-matched is needed to separate architecture from size.

Part 2 - Data and preprocessing

Setup

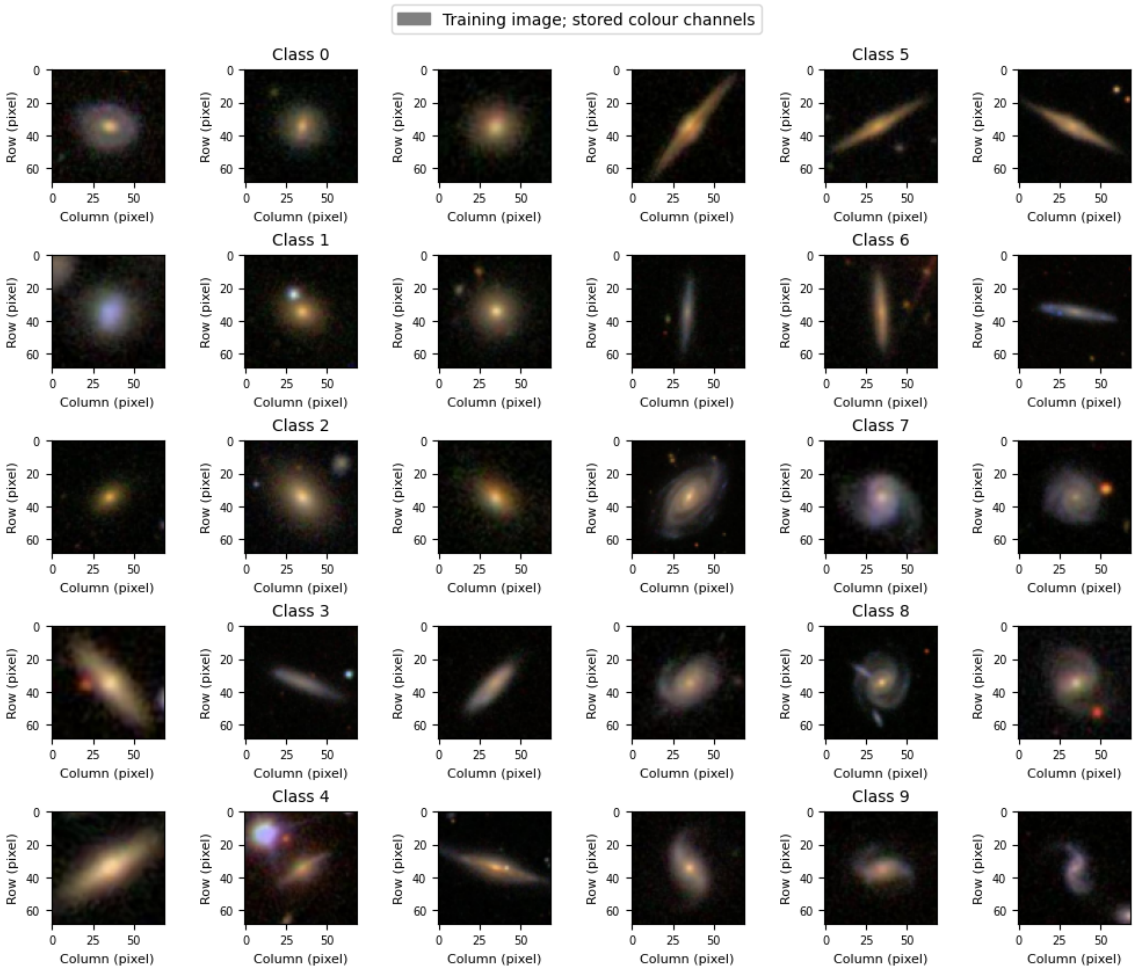
I loaded Galaxy10 SDSS with h5py from the astroNN download.

2.1. Data and split

Images have shape **(21,785, 69, 69, 3)**, type **uint8**, and range **0–255**; labels have shape **(21,785,)**. The grid shows three training images per class.

Class	Description	Images
0	Face-on, no spiral	3,461
1	Smooth, round	6,997
2	Smooth, intermediate	6,292
3	Smooth, cigar	349
4	Edge-on, rounded bulge	1,534
5	Edge-on, boxy bulge	17
6	Edge-on, no bulge	589
7	Tight spiral	1,121
8	Medium spiral	906
9	Loose spiral	519

I kept all classes. Class 5 is rarest; 3, 9, and 6 are also small. A stratified 70/15/15 split (seed **2550**) gives **15,249 / 3,268 / 3,268** images, including **12 / 3 / 2** from class 5. Exploration and preprocessing use only training data; testing starts in Part 3.



Three training images per class.

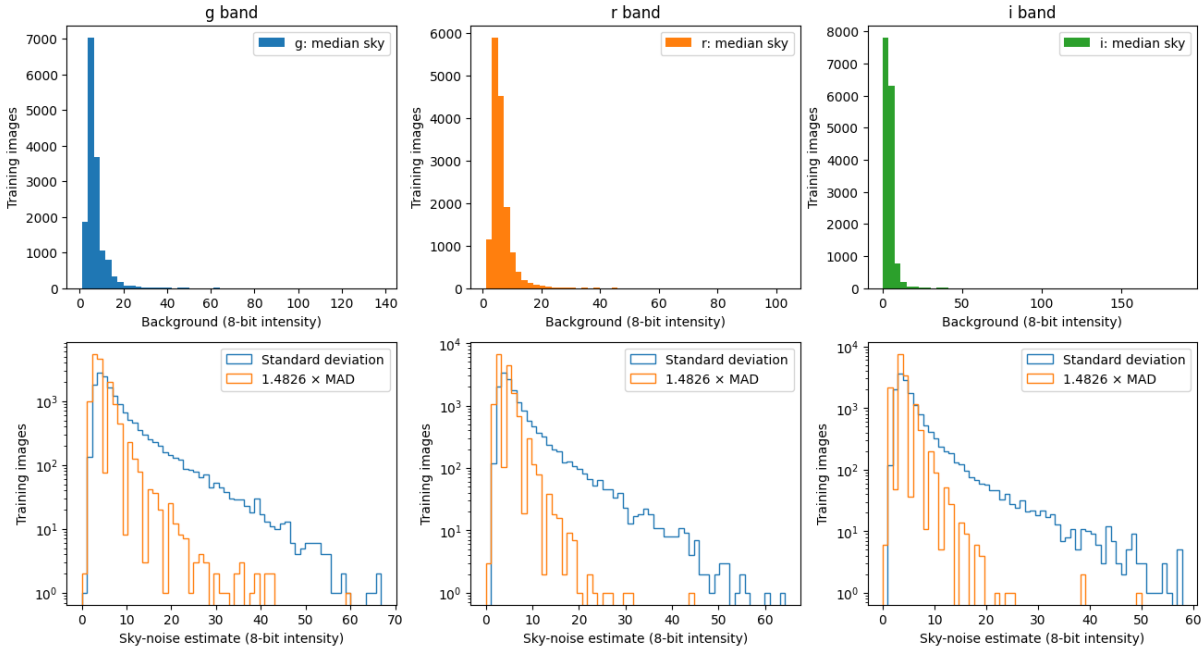
2.2(a). Sky noise

For each training image and band, I used the outer four-pixel frame (1,040 pixels) to find the median sky, standard deviation, and

$$\sigma_{\mathrm{MAD}} = 1.4826\ \mathrm{median}(|I - \mathrm{median}(I)|).$$

Band	Median sky	Median standard deviation	Median MAD noise
g	6	5.99	4.45
r	6	5.21	2.97
i	3	4.51	2.97

Neighbors and galaxy outskirts raise the standard deviation more than the MAD. The middle-band sky's 5th–95th percentile range is 3–12, so one sky value wouldn't suit every image.



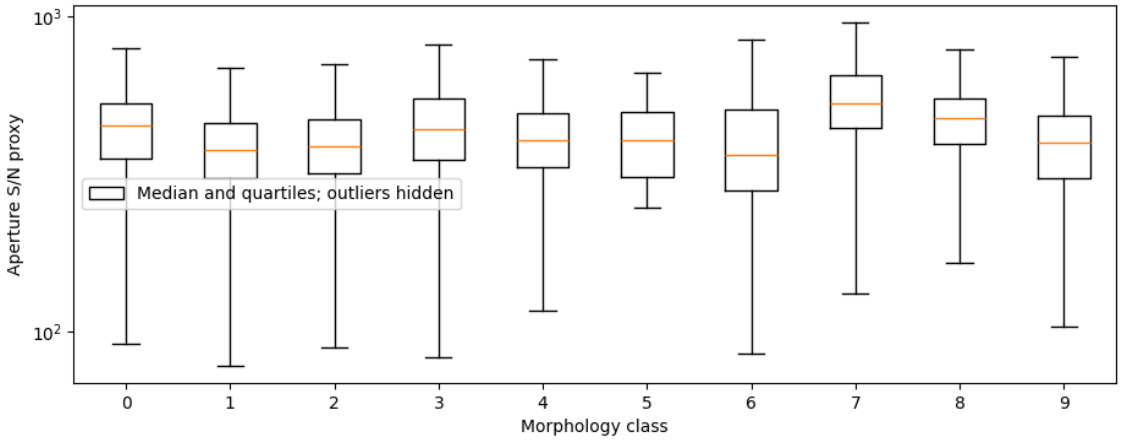
Sky levels above, and noise estimates below.

2.2(b). Signal-to-noise ratio

I used the middle band and a radius-8 aperture centered at $(34, 34)$, containing 197 pixels:

$$\mathrm{S/N} = \frac{\sum_{p \in A}(I_{p,r} - b_r)}{\sigma_{\mathrm{sky},r}\sqrt{197}}.$$

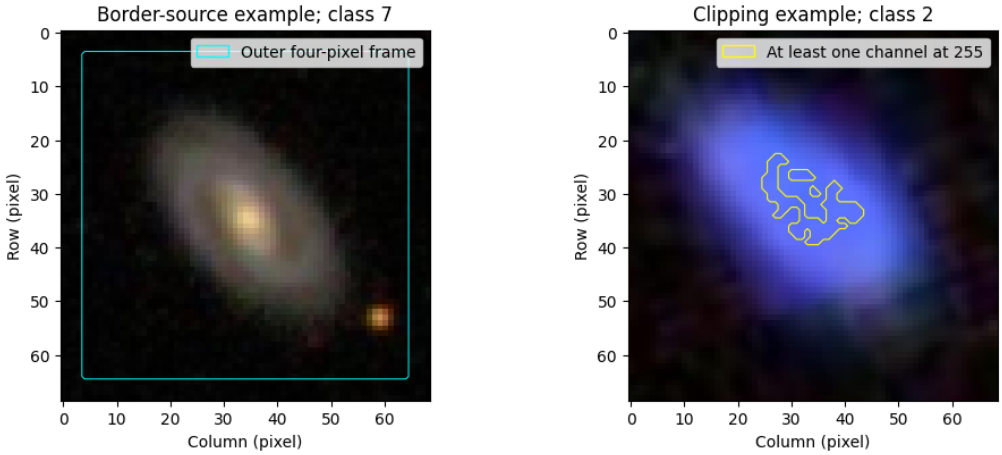
Some MADs are zero, so I set a minimum noise of $1/\sqrt{12}$, the one-count rounding noise. Class 6 has the lowest median S/N (**363.5**) and flux (**12,348**); class 7 has the highest median S/N (**532.0**). Class 6 looks fainter here, though distance and size also matter. This estimate omits shot noise and pixel correlations.



S/N medians and quartiles by class; outliers are hidden.

2.2(c). Contamination and clipping

I flagged three or more touching border pixels above $b_r + 5\max(\sigma_{\mathrm{MAD},r}, 1)$, including diagonals. **61.70%** of images pass; outskirts and noise can also trigger this. **0.001758%** of spatial pixels have a channel at 255. Neighbors bias the sky, while clipping and rounding erase brightness differences. Clipping in these files doesn't necessarily mean detector saturation.



A flagged border source and an example of clipping.

2.2(d). Label noise

I expect confusion between 1/2, 2/3, 4/5/6, and 7/8/9, plus class 0 and smooth galaxies when arms are faint. Volunteers can disagree, so even a sensible prediction can fail against the test label. We would need vote information to estimate an accuracy ceiling.

Cropping, sky subtraction, and averaging can help with unwanted light and noise. They can't restore clipped detail or fix uncertain labels.

2.3(a). Crop

The average sky-subtracted training image has most of its light near the center. Using one-pixel radial bins, the **48 by 48** crop (`10:58`) keeps **91.15%, 91.29%, 90.62%** of positive stacked light in g/r/i. I accepted losing some outskirts to remove border sky and contamination. Large or off-center galaxies can lose more.

2.3(b). Normalization

After converting to float32 and subtracting the sky, I used

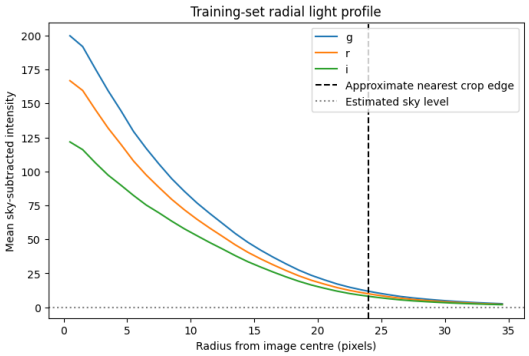
$$z_{p,c} = \frac{I_{p,c} - b_c - \mu_c}{s_c}.$$

Training means are **(38.8693, 32.6744, 26.2158)** and standard deviations **(44.5335, 37.3249, 31.6007)**. After preprocessing, means are within 2×10^{-8} of zero and standard deviations are **1.0000**. I reuse the training values for validation and test to avoid data leakage.

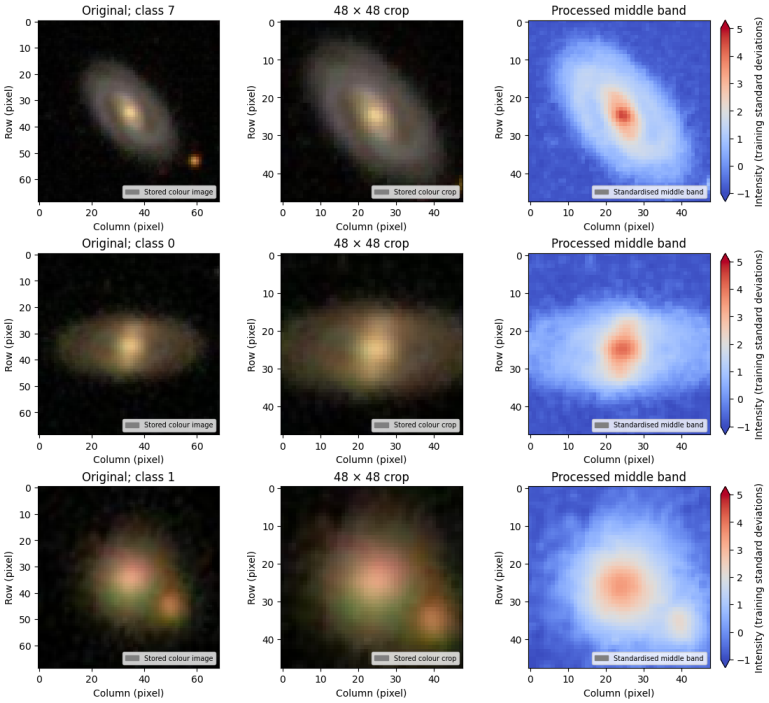
Dividing by each image's brightest pixel removes overall brightness and is sensitive to neighbors. Doing it separately per band also removes color. I kept a shared scale because brightness tells us how visible the galaxy is.

2.3(c). Background subtraction

I subtracted each band's frame median because sky levels vary. Bright pixels affect it less than the mean, though a large galaxy can still bias it high. I used no smoothing or sharpening. The processed examples use a color scale for the middle band because values can be negative.



Average radial profile; the dashed line marks the crop edge.



Original images, crops, and normalized middle bands.

2.3(d). Class imbalance

I used class-weighted cross-entropy, $w_c = N_{\text{train}}/(10n_c)$, and shuffled batches. Plain accuracy favors common classes, so I also report **balanced accuracy**, the average class recall. Weighting gives rare classes more say, but class 5 still has only two test examples.

2.4. Augmentation

Transformation	Use it?	Reason
Arbitrary rotation	Carefully	Same label, but interpolation/cropping can lose detail.
90° rotations	Yes	Same label, no interpolation.
Horizontal/vertical flips	Yes	Labels don't distinguish spiral handedness.
Small shifts	Yes	Position isn't morphology, unless the crop cuts it off.
Zoom	Small amounts	Changes apparent size and visible detail.
Relative band intensity	No random changes	Color reflects stars, dust, and redshift.
Blur	For known seeing	Can erase arms and bulges.

I would start with flips and right-angle rotations. The required baseline uses **no augmentation**.

Part 3 - FCNN and CNN comparison

Training settings

All models use the same data, preprocessing, weighted loss, Adam (**0.001**), batch size **128**, and at most **50 epochs**. Early stopping waits **5** epochs without lower validation loss, then restores the best weights. Baseline seeds are **11, 22, 33**; scrambled runs use **11**. No augmentation or added weight decay.

I timed training and validation on an **Intel Core i7-13700H CPU**, using four threads and PyTorch 2.14.1+cpu. Preprocessing and saving are excluded. The code synchronizes CUDA if a GPU is used.

3.1. Baseline results

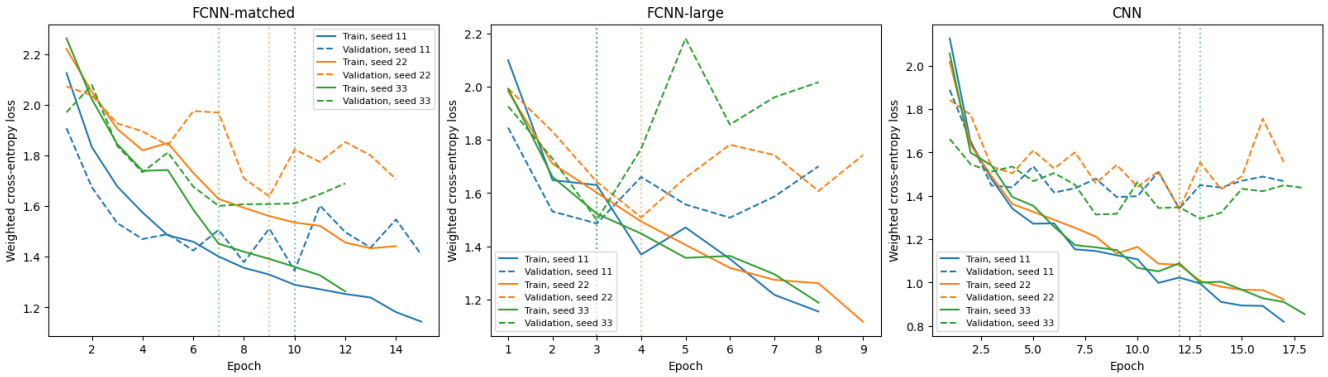
The target is FCNN-matched's best validation accuracy, **61.41%**. The main scores and times are in Part 4(a).

Model	Target reached	Epochs to target	Selected epoch
FCNN-matched	1/3	13.00 (one run)	8.67 ± 1.53
FCNN-large	2/3	6.50 ± 2.12	3.33 ± 0.58
CNN	3/3	12.67 ± 2.52	12.33 ± 0.58

Values are mean $\pm$ sample standard deviation. Target times and epochs count only successful runs and use the first crossing. One successful run has no across-seed deviation. Selected epochs have the lowest validation loss. The code's parameter counts agree with T1.

3.1. Loss curves

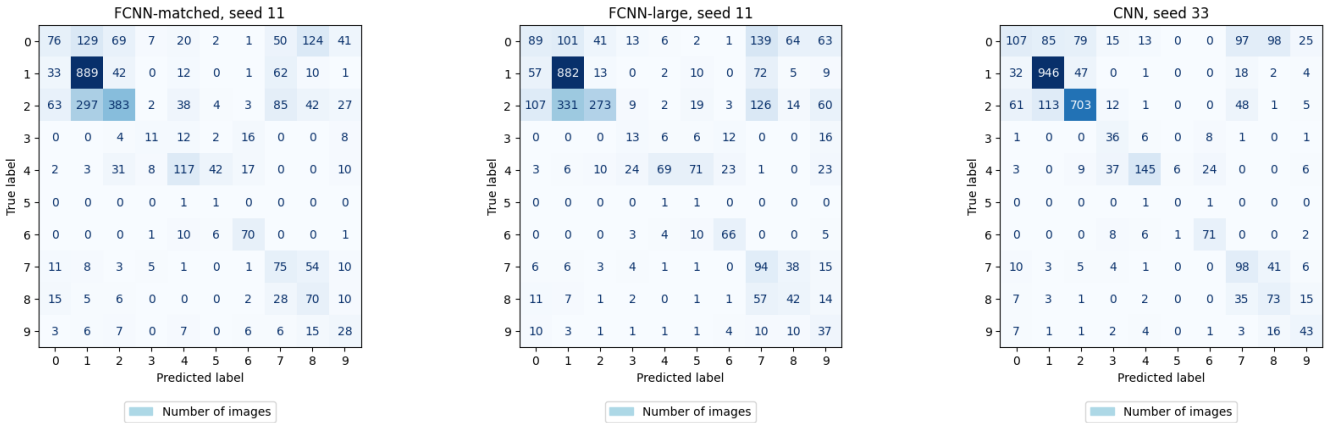
For CNN seed 11, training loss falls **1.024→0.820** over epochs 12–17 while validation loss rises **1.342→1.468**: it overfits too. I kept epoch 12. Each of the three class-5 validation images has weight about **127**, making validation noisy. Dropout is on only during training.



Training and validation loss. Dotted lines mark the restored epochs.

3.1. Confusion matrices

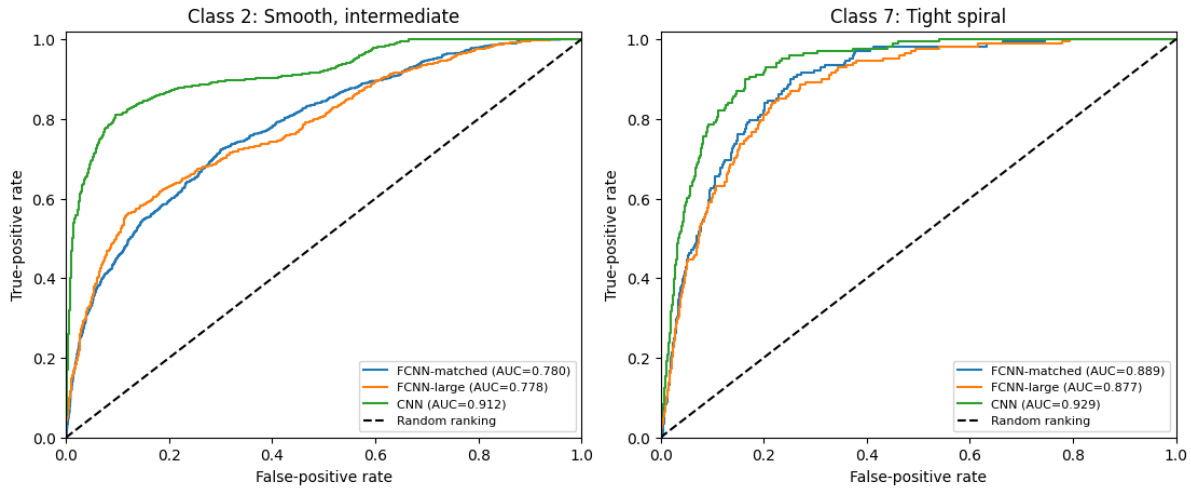
The best seeds by validation loss are **11, 11, 33** for matched, large, and CNN. Their most common mistake is **class 2 as class 1**: **297, 331, 113** images. The CNN swaps 41 tight and 35 medium spirals into each other's classes. These match the expected overlaps. It also misses both class-5 images and gets only **20.6%** of class 0 right.



Best seed of each model. Rows are true classes; columns are predictions.

3.1. ROC curves

Using softmax scores, class-2/class-7 AUCs are **0.780/0.889** (matched), **0.778/0.877** (large), and **0.912/0.929** (CNN). The CNN ranks these classes best; AUC isn't the same as ten-class accuracy.



ROC curves for classes 2 and 7.

3.2. Shifted images

I shifted each crop by $d = 0, 2, 4, 6, 8$ in a random direction (seed **3200**), rounded to pixels. All models use the same shifts, original sky estimates, normalization, and weights.

Model	**Accuracy at $d = 8$ (%)**	**Drop from $d = 0$ (points)**
FCNN-matched	20.25 ± 3.64	30.28
FCNN-large	17.25 ± 3.74	34.70
CNN	30.79 ± 4.85	27.15

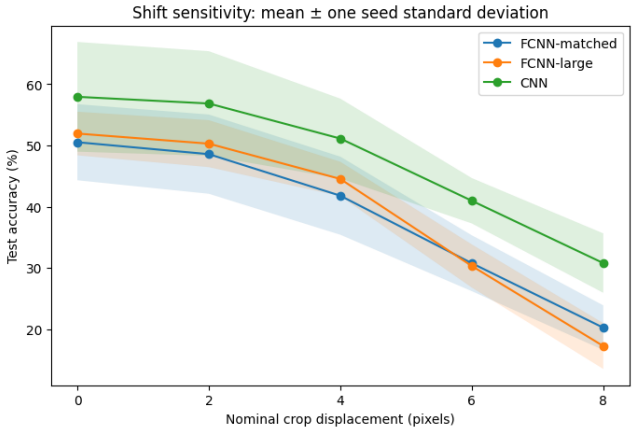
The CNN loses less, as predicted, but isn't invariant. Pooling boundaries, position-specific dense weights, and cut-off light still matter. Rounding slightly changes the distances from d.

3.3. Scrambled-image results

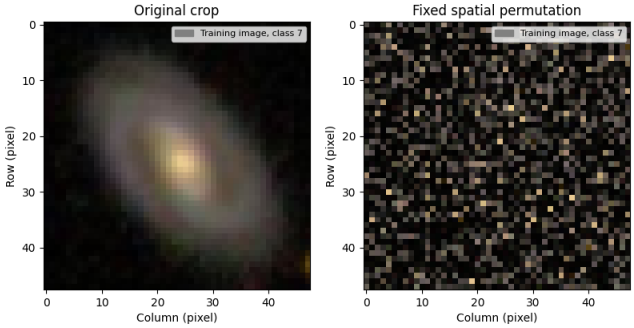
One shuffle (seed **2026**) is shared by training, validation, and test, keeping each pixel's color channels together. Both models are retrained with seed **11** and compared with their seed-11 baselines.

Model	**Original accuracy (%)**	**Scrambled accuracy (%)**	**Change (points)**
FCNN-matched	52.63	50.40	-2.23
CNN	55.05	63.68	+8.63

The FCNN barely changes, as expected. **The CNN improves, so my prediction was wrong.** Its dense layer can still combine shuffled information. One run doesn't show that scrambling helps: baseline CNN scores vary by about nine points, and rare validation images affect early stopping. I would need more seeds and shuffles to check.



Shift accuracy: mean and standard deviation across seeds.



One image before and after shuffling.

Part 4 - Discussion

4(a). Performance and speed

I would pick the CNN for accuracy, but it takes much longer on this CPU. It reaches the target in all three runs, versus one and two for the FCNNs. It isn't faster in wall-clock time.

Model	Accuracy (%)	Balanced accuracy (%)	Parameters	MACs / image	Seconds / epoch	Seconds to target
FCNN-matched	50.53 ± 6.21	44.73 ± 2.26	168,162	168,064	1.61 ± 0.49	27.96
FCNN-large	51.95 ± 3.56	44.72 ± 0.54	1,803,914	1,803,520	2.59 ± 0.36	15.56 ± 5.14
CNN	57.95 ± 8.96	52.04 ± 4.26	171,754	6,451,840	36.26 ± 0.67	466.31 ± 100.69

Mean $\pm$ sample standard deviation; parameters and MACs are fixed. Target times include only successful runs (one for FCNN-matched).

4(b). What about these images helps?

The CNN handled shifts better. The crop kept **91%** of the average image's light, so most structure is near the center. Small filters can find parts of arms and bulges for later layers to combine, though scrambling didn't show this benefit. Noise (**2.97–5.99 counts**) and contamination justify preprocessing; I didn't separately test pooling.

4(c). Five magnitudes

I would use a small FCNN for (u, g, r, i, z): there is no 2D image structure. A 1D CNN assumes patterns can move between bands without changing meaning, which doesn't fit five fixed, unevenly spaced filters. It makes more sense for a densely sampled spectrum.

4(d). What I would change first

I would check labels and get more rare-class examples first. The CNN misses both class-5 images and gets only **20.6%** of class 0 right, despite its reasonable S/N. Noise, clipping, resolution, and other sources limit what is visible. A bigger network can't recover missing detail or settle unclear labels.

AI Usage

I used OpenAI Codex for help with the theory, code, debugging, and writing. The numbers come from running the notebook.